CPU EDUCATION

COMP90051 期末班

□ 4。 ~ 。 讲解

TUTOR：JJ 学长

时长：2 小时

本周内容预览：

1.Feed-Forward Neural Network

2.Autoencoder

3.CNN

4.RNN

CPU EDUCATION

关于CPU EDUCATION

CPU Education成立于2018年，总部在澳大利亚墨尔本，目前国内办公室在广州。经过近多年的积累与发展，公司凭借优质的教学质量和专业的研发团队力量不断壮大，成为全球最受学生欢迎的教育机构之一，在众多学生及老师中享有良好的口碑。CPU Education争做行业标杆，为不同阶段的学生提供专业，专属，专注的教学服务。

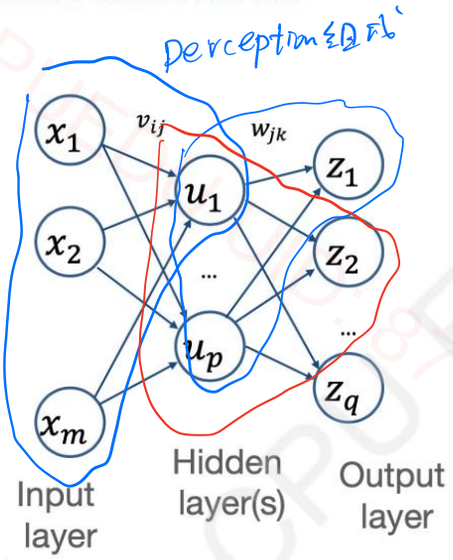
致力于为学生实现人人H1的目标，以“为学生提供极具价值的教学服务，培训各行业精英”为宗旨。秉承“教育高于一切”的经营理念，本着服务定制化、师资专业化、教学规范化的要求，打造墨尔本教育行业的“黄埔军校”。

课后，如果您有任何建议和意见，我们都非常欢迎您联系小助手分享您的想法，给予我们改进和提高的机会!
感谢您参与CPU Education的辅导课程!

课程反馈

PART 1 Feed Forward Neural Network

Feedforward Neural Networks



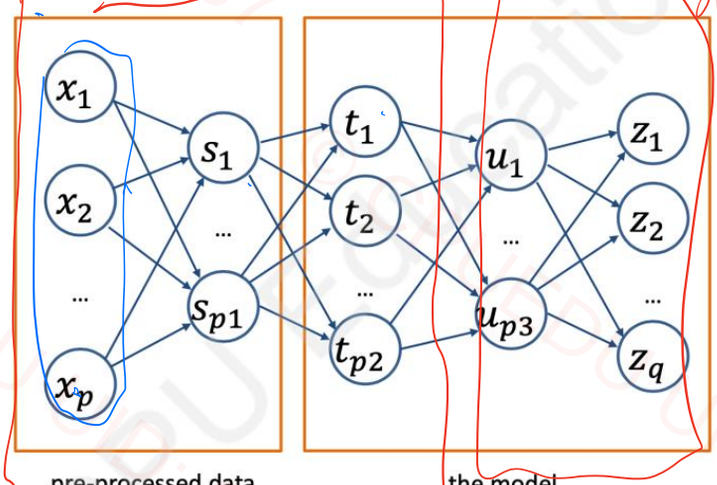
$$r_j = \sum_{i=0}^{m} x_i v_{ij} \qquad u_j = g(r_j)$$

$$s_k = \sum_{j=0}^{p} u_j w_{jk} \qquad z_k = h(s_k)$$

Can deal with non-linear data given

- at least one hidden layer
- non-linear activation function

Feedforward Neural Networks: see layers as "automatic feature extraction"



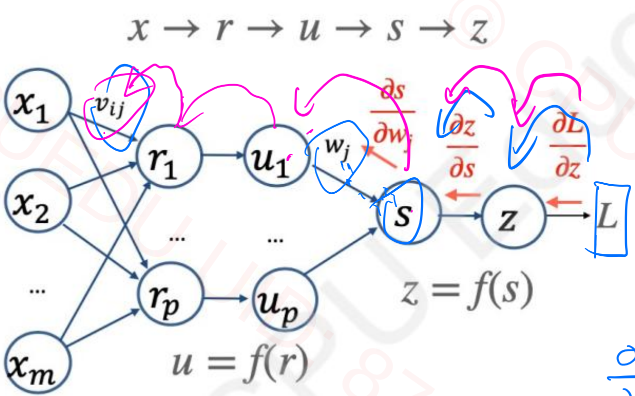
Data transformation is a "learnable function" (with parameters that can be updated in training)

Step function	$f(s) = \begin{cases} 1, & if\ s \geq 0 \\ 0, & if\ s < 0 \end{cases}$
Sign function	$f(s) = \begin{cases} 1, & if\ s \geq 0 \\ -1, & if\ s < 0 \end{cases}$
Logistic function	$f(s) = \dfrac{1}{1+e^{-s}}$
tanh function	$f(s) = \dfrac{e^{s}-e^{-s}}{e^{s}+e^{-s}}$
Rectified linear unit	$f(s) = \max\{0, s\}$

② 当 z 很小时 tanh(z)≈(z)

Feedforward Neural Networks: backpropagation

Use chain rule to compute the partial derivatives of loss function with respect to model parameters



$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z}\frac{\partial z}{\partial s}\frac{\partial s}{\partial w_j}$$

$$\frac{\partial L}{\partial w_j} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial w_j}$$

$$\frac{\partial L}{\partial v_{ij}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial u_j} \cdot \frac{\partial u_j}{\partial r_j} \cdot \frac{\partial r_j}{\partial v_{ij}}$$

2.4 Regularization in Deep Neural Networks (DNNs)

- **Early Stopping**
  - Monitoring validation error during training and stopping when the error stops decreasing.
  - Prevents overfitting by halting training before the model memorizes noise.
  - With some activation functions, early stopping can shrink the DNN towards a linear model.

- **Explicit Regularization** Weight Decay
  - Analogous to Ridge regression, explicit L2 regularization (weight decay) penalizes large weights:

$$L(data, \theta) + \lambda \left(\sum_{i=0}^{m} \sum_{j=1}^{p} v_{ij}^2 + \sum_{j=0}^{p} w_j^2 \right)$$

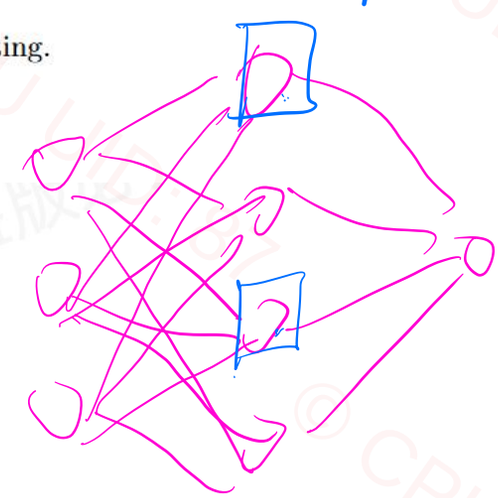
Vij 和 Wj 都惩处

  - This adds $2\lambda v_{ij}$ and $2\lambda w_j$ terms to the gradient, controlling the magnitude of weights and preventing overfitting.
  - With activation functions like tanh/sigmoid, this also encourages the network to behave more linearly.

- **Dropout**
  - Randomly masks a fraction of units during training, forcing redundancy and ensemble-like behavior.
  - Promotes generalization by preventing units from co-adapting.
  - Results in smaller weights and less overfitting.
  - Used in most state-of-the-art deep learning systems.

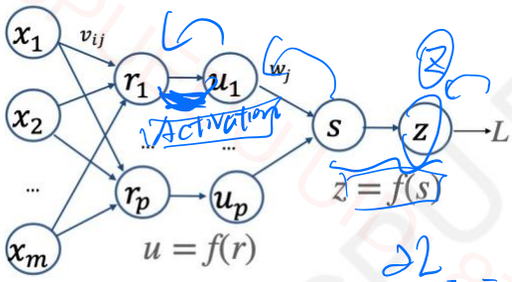
训练中

mask掉的unit 不会更新



最终使用 是用全部

Given the following neural network for binary classification, if we use binary cross entropy as loss function, compute the partial derivative of loss function L with respect to vij (written as a formula that only contains xi, uj, wj, z and y, where y is the correct label of data. Assume we use sigmoid activation function everywhere, which are shown as f in the figure below.)



$f(a) = \frac{1}{1+e^{-a}}$ $f'(a) = f(a)(1-f(a))$

Cross-entropy

$L = -[\, y\ln(z) + (1-y)\ln(1-z) \,]$

$$\frac{\partial L}{\partial v_{ij}} = \frac{\partial L}{\partial z} \cdot \frac{\partial z}{\partial s} \cdot \frac{\partial s}{\partial u_j} \cdot \frac{\partial u_j}{\partial r_j} \cdot \frac{\partial r_j}{\partial v_{ij}}$$

$$\frac{\partial L}{\partial z} = -\left(\frac{y}{z} - \frac{1-y}{1-z}\right)$$

$$\frac{\partial z}{\partial s} = f'(s) = z(1-z)$$

$s = \sum w_j u_j$

$$\frac{\partial s}{\partial u_j} = w_j$$

$$\frac{\partial u_j}{\partial r_j} = f'(r_j) = u_j(1-u_j)$$

$r_j = \sum v_{ij} x_i$

$$\frac{\partial r_j}{\partial v_{ij}} = x_i$$

$$\frac{\partial L}{\partial v_{ij}} = \left(\frac{y}{z} + \frac{1-y}{1-z}\right) \cdot z(1-z) \cdot w_j \cdot u_j(1-u_j) \cdot x_i$$

$y = w_0 \phi_0(x) + w_1 \phi_1(x) + \cdots$

4. Explain how *artificial neural networks* can be considered to be a form of non-linear *basis function* when learning a linear model. [4 marks]

$h_j(x) = \sigma(w_j^T x + b_j)$ Non linear basis

ANN 的每个隐藏单元都对 6 这个非线性激活函数进行变换，变换后

ANN 输出层 $\hat{y} = \sum v_j h_j(x)$

所以它相当于 Non-linear basis 的 linear model

Question 13: Neural networks & Regularisation [10 marks]

a) Consider the training objective, $\mathcal{L}(\theta) = -\log P_\theta(\mathbf{y}|\mathbf{X}) + \lambda r(\theta)$, where the second term is a *regulariser*, $\lambda > 0$ and $r(\theta) \geq 0$ is a positive-valued function. For the minimiser of the regularised training objective to be a *maximum a posteriori* estimate, state the general form of the corresponding prior over θ. [4 marks]

b) Given that the output loss of a *feed-forward network*, L, is not a direct function of *hidden-layer weights*, $\mathbf{w}$, how is the derivative of loss with respect to hidden-layer weights, $\frac{\partial L}{\partial \mathbf{w}}$, calculated in *backpropagation*? As part of your answer include the formulation of $\frac{\partial L}{\partial w_j}$ where j is the index of a single weight parameter. [6 marks]

a). Target $\min L(\theta) = -\log P_\theta(y|x) + \lambda r(\theta)$

$= \max -L(\theta) = \log P_\theta(y|x) - \lambda r(\theta)$

$\Leftrightarrow \log\left(P_\theta(y|x) \cdot e^{-\lambda r(\theta)}\right)$

MAP: $\text{Arg max}\, P(\theta|X,y) \propto P(y|x;\theta) \cdot P(\theta)$

$P_\theta(y|x) \cdot e^{-\lambda r(\theta)} \propto P(y|x;\theta) \cdot P(\theta)$

$P(\theta) \propto e^{-\lambda r(\theta)}$

b). 由于 L 不直接依赖于 w_{ij}，而是依赖于隐藏层 h_j

$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial w_{ij}} = \frac{\partial L}{\partial h_j} \cdot \frac{\partial h_j}{\partial z_j} \cdot \frac{\partial z_j}{\partial w_{ij}}$

Assume $h_j = \sigma(z_j)$ $z_j = \sum_i w_{ij} x_i + b_j$

$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{h_j} \cdot \sigma'(z_j) \cdot x_i$

(a) Consider training a *deep neural network* with *tanh activation functions* using *backpropagation*. Can this lead to the *vanishing gradient problem*? Why? [5 marks]

Yes

① 在DNN中，梯度通过 chain rule 传播

eg $\frac{\partial L}{\partial \theta} = \frac{\partial L}{a^{(n)}} \cdot \frac{\partial a^{(n)}}{a^{(n-1)}} \cdots \frac{\partial a^{(1)}}{\partial \theta}$

但对于 tanh 作为激活函数，tanh'(x) 容易出现导数趋于0 (输入饱和) tanh'(x) ≈ 0，而一堆趋于0的乘积会使前面整个梯度趋于0，这就是 vanish gradient

(b) Consider training a *deep neural network* for binary classification. Assume *tanh activation functions* on the hidden layer and training with *backpropagation*. How might this lead to the *vanishing gradient problem*? [3 marks]

(c) Discuss how a neural network with *tanh activation functions* in its hidden layers can be made into an effectively linear model through regularization. [4 marks]

w_i 变小

$z = \sum w_i x_i$

输入 z 很小时 $\tanh(z) \approx (z)$

weigh Decay，惩罚权重过大项

让 NN 权重尽量小

$z = \sum w_i x_i$ 也会变得很小

在 tanh 中，当 z 很小时 $\tanh(z) \approx z$

这就意味着模型变成 linear model through regularisation

(d) Consider training a *deep neural network* for predicting temperature given humidity. Which *output* activation would you use and why? Why would *relu* not be appropriate? [3 marks]

temperature 回归问题

线性激活函数

Relu = max(0, s)

输出恒非负，不适合用于预测温度

(e) While training a neural network model, you observe that the training error initially decreases and but then increases and seems to converge. What has occurred? Provide 2 mechanisms for mitigating this situation and explain why they work. [5 marks]

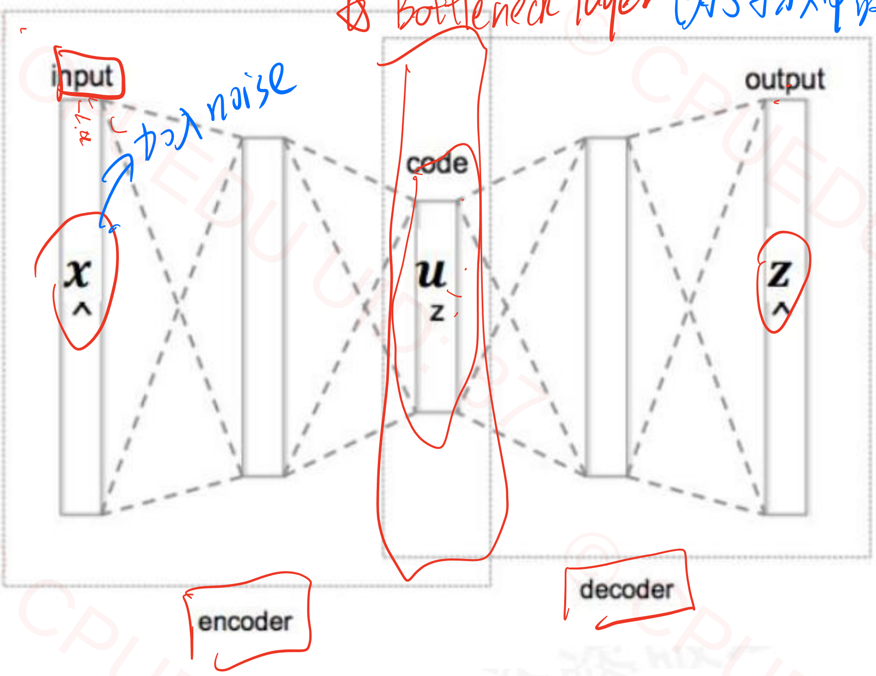
过拟合

① Early stopping

② Weight Decay (L2正则化)

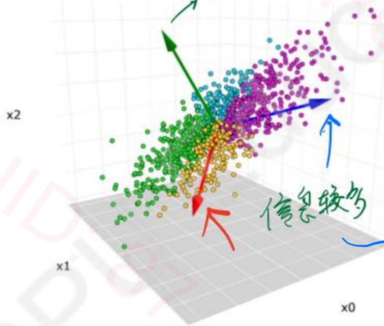
③ Dropout

PART 2 Autoencoder



Autoencoder vs Principle Component Analysis

- Both can be used for dimensionality reduction.
  - ➢ PCA finds orthonormal basis where axes are aligned to capture maximum data variation
    - which are eigenvectors of dataset covariance matrix



➢ Autoencoders (based on neural network-based encoder and decoder) don't explicitly use eigenvalues / eigenvectors

9. With respect to learning an *autoencoder*, explain a failure case that can arise from the use of complex non-linear encoder and decoder components. [4 marks]

过拟合

因为复杂的 非线性 Encoder 与 decoder 的学习能力很强，容量很大，网络只是在 copy input，导致了过拟合，没有提取到泛化特征，从而泛化性能差

What is the difference between an undercomplete autoencoder and an overcomplete autoencoder?

undercomplete

bottleneck layer dimension < input data

purpose 为 to compress input to capture most important feature

overcomplete

dimension > input data

due to large capacity, he is only copying input

What problem does a denoising autoencoder aim to solve, and how does it achieve this?

解决过拟合，提升模型泛化

how? 给 input data 添加 Noise (corrupt)

然后 encode，尝试重建 uncorrupt data

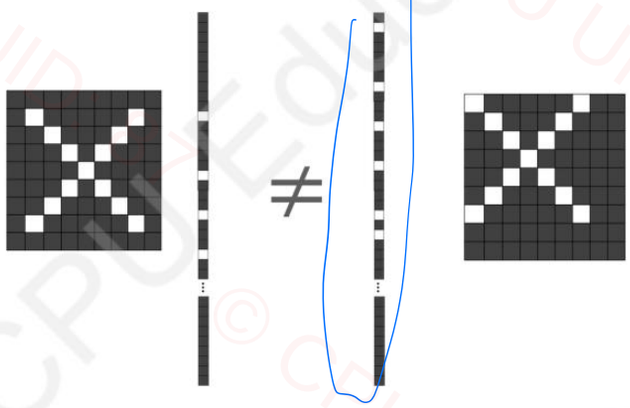
用

PART 3 CNN

Convolutional Neural Network

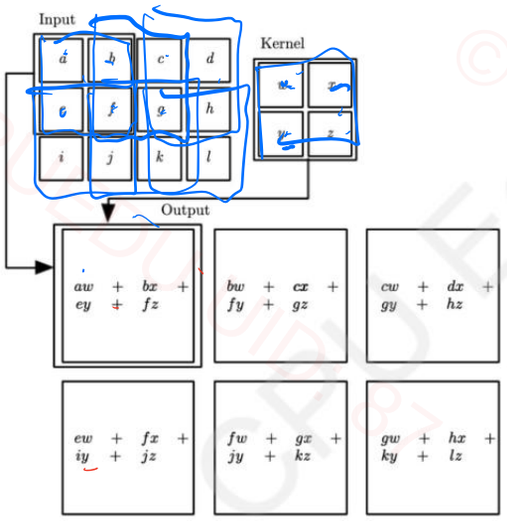
Designed for 2-D inputs (image)

- Key advantage compared to Feedforward Neural Networks: Translation invariance



Convolutional Neural Network

Designed for 2-D inputs (image)



convolution
- kernel size
- stride
- padding

$$H_{out} = \left\lceil \frac{H_{in} + 2 \times h_{padding} - h_{kernel} + 1}{stride_h} \right\rceil$$

$$W_{out} = \left\lceil \frac{W_{in} + 2 \times w_{padding} - kernel + 1}{stride} \right\rceil$$

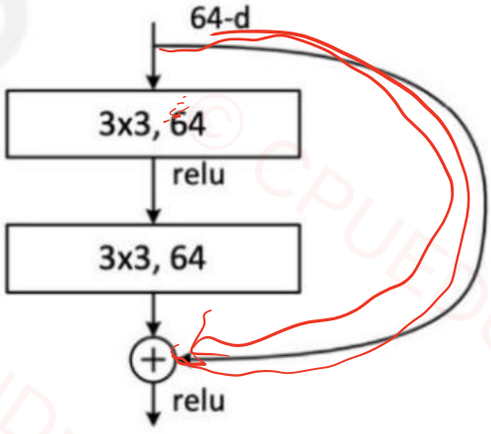
Convolutional Neural Network

Designed for 2-D inputs (image)

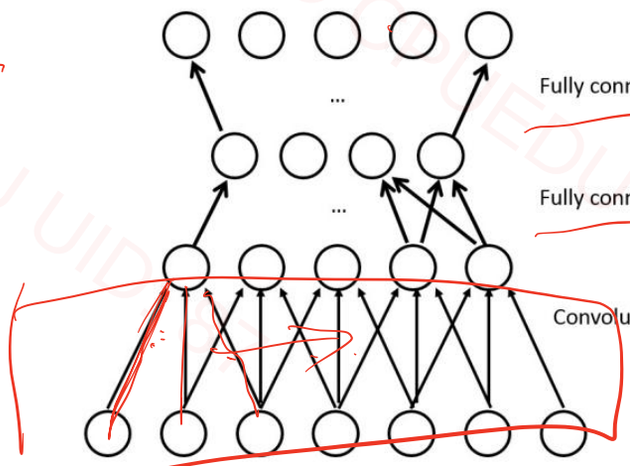
pooling
- kernel size
- stride

common pooling methods
- max pooling
- average pooling

Residual Connection



(d) How many *parameters* does the following *convolutional neural network* have (exclude the *bias*)? **[3 marks]**



4×5 Matrix

5×4 matrix

3×1 matrix

5×4=20

5×4=20

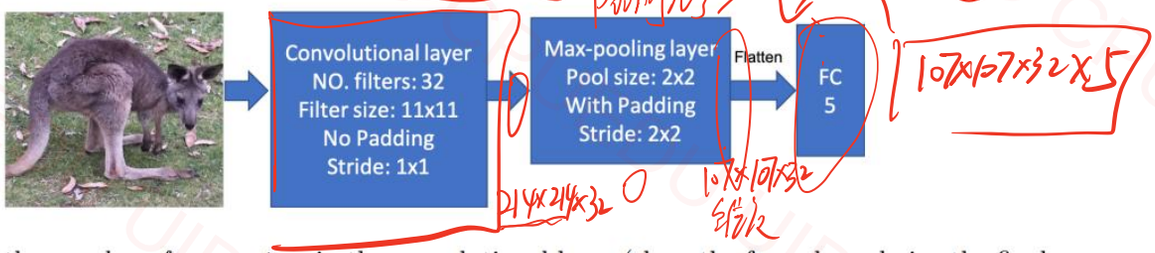
多长步子

3个参数

3+5×4+4×5=43

Question 6: Convolutional Neural Network [8 marks]

Consider the following simple *Convolutional Neural Network*. The input is a *RGB image (3 channels)* with width and height equal to 224. The setting of the layers are shown in the figure. There are 32 filters in the convolutional layer and each filter has a size of 11×11.



(a) Compute the *number of parameters* in the convolutional layer (show the formula and give the final result) [3 marks]

(b) In order to reduce the number of parameters of the convolutional layer in the figure, while *keeping the receptive field the same*, one possible method is to stack one 3×3 convolutional layer and one 9×9 convolutional layer. Describe five different possible stackings, i.e., different combinations of multiple (can be two or more than two) convolutional layers. (Note: changing the order of the multiple convolutional layers is counted as the same stack, e.g., "Stacking a 9×9 and a 3×3 convolutional layer" is counted as the same stack as "Stacking a 3×3 and a 9×9 convolutional layer".) [5 marks]

Question 5: Convolutional Neural Networks [14 marks]

Consider a simple *Convolutional Neural Network* for processing 3×3 images, with a single *filter* of size 2×2, applied with no *padding* and *stride* of 1×1. The next step is *global max-pooling*, which takes the *filter* output (size 2×2) to produce a single output.

1. What mathematical function is defined by the network, relating its output y to its input $x_{1,1}, x_{1,2}, \ldots, x_{3,3}$? You will need to introduce new variables for the model's parameters. [6 marks]
2. Explain how a *loss gradient* at the output is *back-propagated* through the network, and show the mathematical form of the loss gradient with respect to the model parameters. [8 marks]

1).

Image : $\begin{bmatrix} x_{11} & x_{12} & x_{13} \\ x_{21} & x_{22} & x_{23} \\ x_{31} & x_{32} & x_{33} \end{bmatrix}$ Assume filter $\begin{bmatrix} w_{11} & w_{12} \\ w_{21} & w_{22} \end{bmatrix}$

Kernel output

$$\begin{bmatrix} y_{11} & y_{12} \\ y_{21} & y_{22} \end{bmatrix} = \begin{bmatrix} w_{11}x_{11} + w_{12}x_{12} + w_{21}x_{21} + w_{22}x_{22} & , \; w_{11}\cdot x_{12} + w_{12}\cdot x_{13} + w_{21}\cdot x_{22} + w_{22}x_{23} \\ \cdots & \cdots \end{bmatrix}$$

$$y = \max\{y_{11}, y_{12}, y_{21}, y_{22}\}$$

2).

$$\frac{\partial L}{\partial w_{ij}} = \frac{\partial L}{\partial y} \cdot \frac{\partial y}{\partial w_{ij}}$$

Gradient backpropagate only works at maximum index i^*, j^*

$$\frac{\partial L}{\partial y_{i^*j^*}} \cdot \frac{\partial y_{i^*j^*}}{\partial w}$$

① if $i^*=1, j^*=1$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial y_{11}} \cdot \frac{\partial y_{11}}{\partial w_{11}} = \frac{\partial L}{\partial y_{11}} \cdot x_{11}$$

$$\frac{\partial L}{\partial w_{12}} = \frac{\partial L}{\partial y_{11}} \cdot \frac{\partial y_{11}}{\partial w_{12}} = \frac{\partial L}{\partial y_{11}} \cdot x_{12}$$

⋮

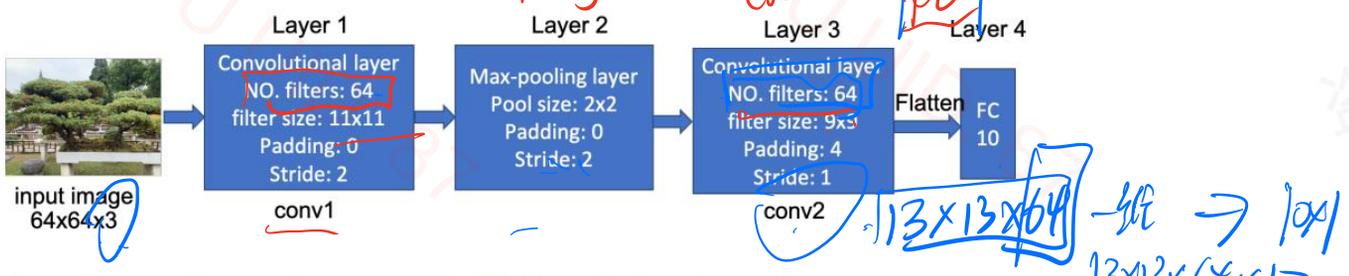
② if $i^*=1, j^*=2$

⋮

Question 5: CNN Architectures [16 marks]

Assume that you train a *convolutional neural network (CNN)* on a GPU. The size of each input image is 64x64x3 (3 channels). The following figure shows the settings of the layers in the CNN. As shown, there are four layers (blue blocks) in the CNN. Specifically, there are two *convolutional layers* (conv1 and conv2). Between them there is a *max-pooling layer*. The output feature map of conv2 is flattened into a vector, which is then fed to a *fully-connected layer (FC)*. The FC layer contains 10 output units.

Note: The padding size is the number of rows/columns on each size of the input. For example, if the size of an input is 5×5 *and the padding size is 1, then after padding, each dimension of the input becomes* $5 + 2 \times 1 = 7$*, so the input becomes* 7×7*.*



(a) Write the sizes of output feature maps of the Layer 1, Layer 2 and Layer 3 (write in the form 'width × height × channels') [6 marks]

(b) How many parameters in *each of the four layers* (exclude the bias) in the CNN? Show your working. [4 marks]

(c) How to replace conv2 to keep the same size of *receptive field* of conv2 with fewer parameters? Write three different replacements. [6 marks]

a).

Layer 1 $\left\lceil \frac{64 + 2\times 0 - 11 + 1}{2} \right\rceil = 27$

output size $27 \times 27 \times 64$

Layer 2 $\left\lceil \frac{27 + 2\times 0 - 2 + 1}{1} \right\rceil = 13$

output size $13 \times 13 \times 64$

Layer 3 $\left\lceil \frac{13 + 2\times 4 - 9 + 1}{1} \right\rceil = 13$

output = $13 \times 13 \times 64$ (channel 数)

b. Layer1 = $11 \times 11 \times 3 \times 64 + 0$

Layer2 = 0

Layer3: $9 \times 9 \times 64 \times 64 + 0$

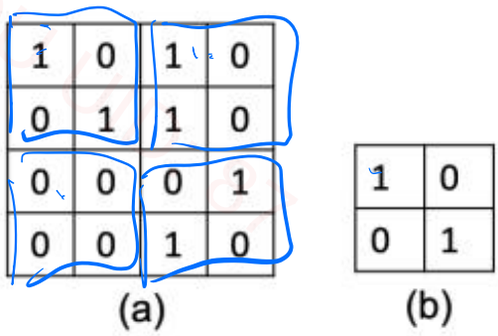
FC: 输入 $13 \times 13 \times 64$

输出 10

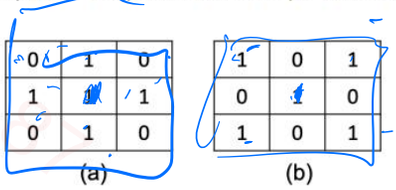
$13 \times 13 \times 64 \times 10$

Question 4: Convolutional Operation [10 marks]

(a) Assume that in the following figure (a) is an *image patch* and (b) is a *kernel* that is used to perform *convolution* (*stride = 2, no padding*) on the patch. Show the *output feature map* after convolution. [4 marks]



(b) Assume that (a) and (b) in the following figure are two image patches. Now consider a 3×3 *binary weight matrix* (each element of the matrix is either 0 or 1) that is used as the kernel to perform convolution (*stride = 1, no padding*) on the two image patches. Assume that after convolution, the output feature maps are K_a and K_b. What kernel can make the difference between K_a and K_b most significant (sum of all elements of $|K_a - K_b|$ is maximum)? List two such kernels. [6 marks]



Question 5: Convolutional Neural Networks [14 marks]

Consider the following toy *convolutional neural network (CNN)*: a single-channel 32×32 pixel input image (first blue square) is first fed into a *convolutional layer* (first black square) resulting in a 15×15 output (second blue square), which is then fed into a *max-pooling layer* (second black square) resulting in an output (third blue square) of unknown dimension.

Prior to the *convolutional layer*, each of the four sides of the image is *padded* by 1 pixel. This layer has a single 5×5 *filter/kernel* with some unknown *stride*. The *max-pooling* layer uses a 2×2 *pool size*, with 2×2 *stride* and no *padding*.

(a) What is the *convolutional layer's* missing *stride* value? [8 marks]

(b) What is the missing final *feature map dimension*? [6 marks]

a). $\left\lceil \frac{32 + 2\times1 - 5 + 1}{\text{stride}} \right\rceil = 15$

$\left\lceil \frac{30}{\text{stride}} \right\rceil = 15$ stride = 2

b). $\left\lceil \frac{15 - 2 + 1}{2} \right\rceil = 7$

7 × 7

(c) Explain how *convolutional neural networks (CNNs)* can produce *translation invariant* representations when applied to vector or matrix inputs. [5 marks]

① Convolution operation, 参数共享
允许 CNN 在不同的 location 检测到相似的特征
(如果因为图像平移

② Pooling operation
aggregate features within small local region
so small shifts in input cause no changes in output

(b) What are two benefits of *max pooling*. [5 marks]

① can get translation invariation features from input

② reduce dimension of feature map

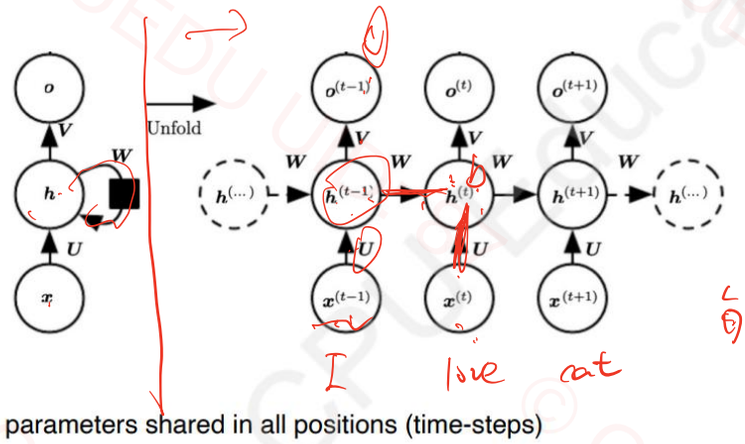
(c) *Padding* can be used to make the output and input dimensions of a *convolutional layer* the same. What is another use of *padding*? [5 marks]

better represation for pixels in corner of the image

PART 4 RNN

Sequence Model: Recurrent Neural Network

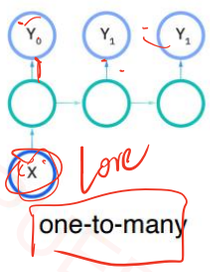
For processing sentence (a sequence of words), audio clip (sequence of time steps) and video clip (sequence of frames)



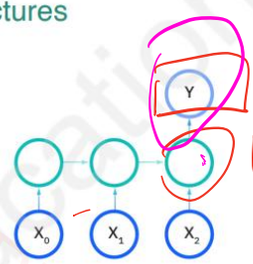
parameters shared in all positions (time-steps)

$$\begin{aligned} \boldsymbol{a}^{(t)} &= \boldsymbol{b} + \boldsymbol{W}\boldsymbol{h}^{(t-1)} + \boldsymbol{U}\boldsymbol{x}^{(t)}, \\ \boldsymbol{h}^{(t)} &= \tanh(\boldsymbol{a}^{(t)}), \\ \boldsymbol{o}^{(t)} &= \boldsymbol{c} + \boldsymbol{V}\boldsymbol{h}^{(t)}, \end{aligned}$$

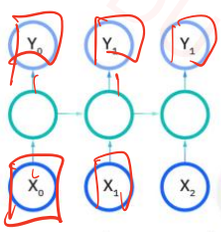
Sequence Model: Recurrent Neural Network Architectures



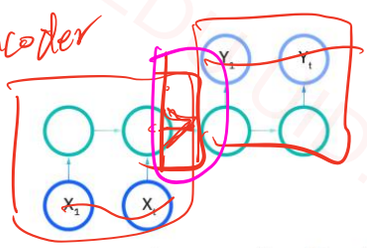
one-to-many



many-to-one (sequence classification)



many-to-many (sequence tagging)

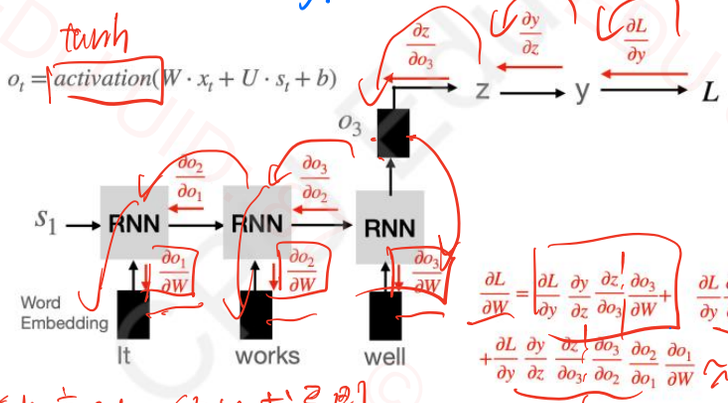


many-to-many (seq2seq)

Sequence Model: Recurrent Neural Network

Use backpropagation through time (BPTT) to update parameters (propagate gradients back to the very start of the network.

Gradient vanishing/explosion problem: if too many time-steps, gradients can be very small / large as a result of multiple multiplications. Can't learn long distance phenomena (10+ steps).



$$\frac{\partial L}{\partial W} = \frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial o_3}\frac{\partial o_3}{\partial W} + \frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial o_3}\frac{\partial o_3}{\partial o_2}\frac{\partial o_2}{\partial W} + \frac{\partial L}{\partial y}\frac{\partial y}{\partial z}\frac{\partial z}{\partial o_3}\frac{\partial o_3}{\partial o_2}\frac{\partial o_2}{\partial o_1}\frac{\partial o_1}{\partial W}$$

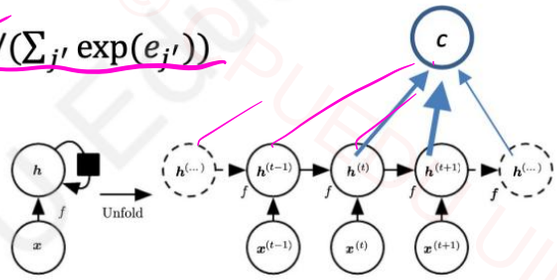
Drawback: RNN只用 last state 来完成, 丢了前面的信息

Solution:

Sequence Model: Attention

Take a weighted average of hidden sequence instead of focusing on later steps

- $\boldsymbol{c} = \sum_j \alpha_j \boldsymbol{h}^{(j)}$
- $\alpha_j = \exp(e_j)/(\sum_{j'} \exp(e_{j'}))$
- $e_j = f(\boldsymbol{h}^{(j)})$



Sequence Model: Attention in Seq2Seq models

Avoids information bottleneck problem

最终输出

$$c_i = \sum_j \alpha_{ij} h^{(j)}$$

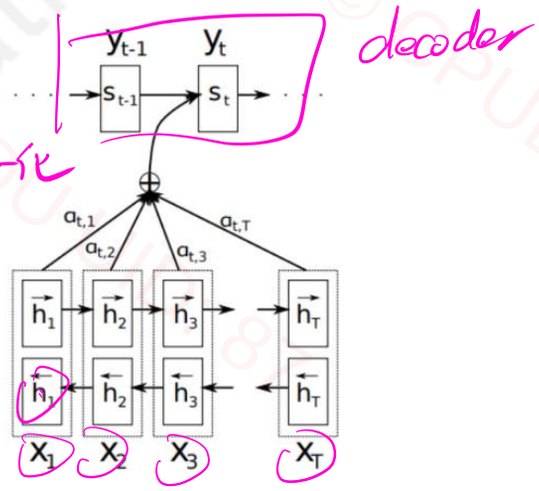
对相关性得分的归一化

$$\alpha_{ij} = \exp(e_{ij}) / (\sum_{j'} \exp(e_{ij'}))$$

$$e_{ij} = f(s^{(i)}, h^{(j)})$$

相关性得分

计算当前位置和第 j 个位的相关性得分



Attention 特点

① 并发性计算

② 最终输出是所有 hidden state 加权组合

1. A *recurrent neural network* is as a type of *deep artificial neural network*. In what respect is it *deep*? [4 marks]

RNN 通过对时间步的重复计算可以被看作 Deep

在 RNN unroll 时间维度，相当于一个 DNN

即只有一层 RNN，如果序列长度增加，隐藏状态

就会经过多次非线性计算，体现了深度。

(d) For the *recurrrent neural network (RNN)* that takes a sequence as the input, explain why we need to use *backpropogation through time* to update weights. [5 marks]

RNN 的输出不仅依赖于当前时间步输入，还依赖于前面隐藏状态，

并且权重是所有时间步共享的

(b) Both *convolutional networks (CNNs)* and *recurrent networks (RNNs)* can be applied to sequence inputs. Explain the key benefit that RNNs have over CNNs for this type of input. [5 marks]

RNN 使用时间步依赖性捕获序列中每个时间步的上下文信息，通过 Hidden State 建模长期依赖

而 CNN 主要通过卷积捕获局部特征，难以获得序列远距离依赖关系

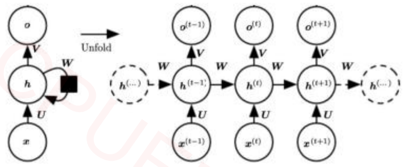
(c) Explain in words how *Attention* can be used to allow for neural models to process dynamic sized inputs. [5 marks]

Attention 首先计算每个位置的相关性得分和归一化再全局加权求和，在整个过程中不需要固定输入，无论输入长度怎么变，整个过程都一样，只是途中的维度有改变

(b) Why are *vanishing gradients* a more pressing concern for *recurrent neural networks (RNNs)* than for other neural architectures? [5 marks]

BPTT，需要在序列的每个时间步都对参数做 back propagate
而每个时间步梯度又要用 chain 乘起来，
而普通序列又比较长导致结构深，
所以容易产生 Vanish Gradient，导致 RNN 在前面时间步难以学习长期依赖关系

How many parameters are there in total? Suppose the dimension of x, h and o are d1, d2 and d3, respectively.



(把矩阵加到一起也的行)

$$\begin{aligned} a^{(t)} &= b + W h^{(t-1)} + U x^{(t)}, \\ h^{(t)} &= \tanh(a^{(t)}), \\ o^{(t)} &= c + V h^{(t)} \end{aligned}$$

R^{d_2}, R^{d_3}, R^{d_2}

$x^{(t)} \in R^{d_1}$

$h^{(t)} \in R^{d_2} \Rightarrow a^{(t)} \in R^{d_2}$

$V \in R^{d_2 \times d_1}$

对 $h^{(t)} \in R^{d_2}$

$W \in R^{d_2 \times d_2}$

$b \in R^{d_2}$

$V \in R^{d_3 \times d_2}$

$c \in R^{d_3}$

$d_2 \times d_1 + d_2 \times d_2 + d_2 + d_3 \times d_2 + d_3$

Why is the Transformer model able to process sequences in parallel, unlike RNNs?

why is attention process in parallel in RNN?

因为在计算相关性得分 $e_{ij} = f(x, h)$ 是并行的
不像 RNN，每一个隐藏状态 vector 都依赖于
previous time step

CPU EDUCATION